

ZeroToken Router

5-Layer Hybrid ML Agent

Revolutionizing General-Purpose AI Agent Cost-Efficiency

The Problem: Skyrocketing AI Costs

- General-purpose agents blindly send **all** queries to expensive cloud APIs.
- "Easy" queries (like simple math or sentiment) waste massive amounts of tokens.
- **Latency** increases exponentially as network round-trips stack up.
- **Goal:** How can we build an agent that is 100% accurate but uses a fraction of the tokens?

Our Solution: The 5-Layer Architecture

We built a highly optimized routing agent that strictly funnels queries through five defensive layers:

1. **Regex Pruning**
2. **Semantic Caching**
3. **XGBoost Intent Routing**
4. **Local Qwen 1.5B Fallback**
5. **Premium API Shunting (Fireworks AI)**

Layer 1 & 2: Instant Cost Savings

1. Regex Pruning

- Strips conversational filler, politeness ("please", "can you"), and whitespace.
- Reduces token overhead drastically *before* processing even begins.

2. Semantic Caching (MiniLM)

- Converts incoming queries into dense vector embeddings.
- If Cosine Similarity > **0.85** against a previously solved task, we return the cached response.
- **Cost: 0 Tokens.**

Layer 3 & 4: True Edge AI

3. XGBoost Intent Router

- A trained Machine Learning model classifies the intent of the prompt (Math, Logic, Sentiment, Code).
- Executes locally on the CPU in `< 5ms`.

4. Local Fallback Engine (Qwen 1.5B)

- "Easy" categories (Sentiment, Factual) are routed to a local **Qwen2.5-1.5B-Instruct** model via `llama.cpp`.
- Executes entirely inside the Docker container without internet.
- **Cost: 0 Tokens.**

Layer 5: Premium API Shunting

- Only the absolute hardest tasks (Complex Code, Advanced Math) reach the premium API.
- We utilize the blazing-fast **Fireworks AI** API.
- We strictly enforce **Chain-of-Draft** prompting to ensure the model outputs the minimal number of tokens required to solve the problem.

The Results

- **Accuracy:** Reached **19/19 (100%)** on the strict judge evaluator script.
- **Token Efficiency:** Routed ~60% of test queries locally or to cache, saving thousands of tokens.
- **Robustness:** Handles network failures with automatic fallbacks and local execution.
- **Deployment:** Fully dockerized, headless, and runs seamlessly on standard CPUs.

Thank You

Ready to deploy the future of token-efficient AI.